49. Redis: What It Is and How to Use It (with NestJS)

1. Redis in simple words

Redis is a very fast store that keeps data in memory (RAM). It saves data as **key** → **value** pairs, like a big JavaScript object that lives on a server.

```
"user:1"    →  '{"id":1,"name":"Gourish"}'
"otp:9876"  →  "4521"
"visits"    →  "1520"
```

> **Real-life example:** Postgres is like a **library**: it holds everything, permanently, but finding a book takes time. Redis is like the **notebook on your desk**: it's small, but you can read it instantly. You copy the things you need often from the library into the notebook.

2. Redis vs Postgres

	Redis	**Postgres**
Stores data in	RAM (memory)	Disk
Speed	Very fast (under 1 ms)	Slower (a few ms or more)
Data shape	Key → value	Tables, rows, columns
Relations / JOIN	No	Yes
Permanent?	Can be lost on restart (unless persistence is on)	Yes
Size	Limited by RAM	Large
Role	**Helper** (cache, temporary data)	**Main database** (source of truth)

> Redis does **not replace** Postgres or MongoDB. It sits **next to** them to make things faster.

3. What Redis is used for

Use	Example
Cache	Save the result of a slow DB query so you don't run it again
Expiring data	OTPs or password reset tokens that delete themselves after 5 minutes
Sessions	Login sessions shared across several servers
Rate limiting	"Max 10 requests per minute per user"
Counters	Page views, likes
Queues	Background jobs like sending emails (BullMQ)
Pub/Sub	Real-time messages between services or for chat

4. Data types (most common)

Type	Think of it as	Example command
String	A single value	`SET name "Gourish"`
Hash	An object with fields	`HSET user:1 name "Gourish" age 25`
List	An array (queue)	`LPUSH tasks "email"`
Set	Unique values	`SADD tags "node" "nest"`
Sorted Set	Values with a score (leaderboard)	`ZADD scores 100 "gourish"`

5. Step 1: Run Redis with Docker

```
docker run -d --name redis -p 6379:6379 redis:7
```

Or add it to `docker-compose.yml` next to Postgres:

```
services:
  postgres:
    image: postgres:16
    # ...your existing config

  redis:
    image: redis:7
    container_name: redis
    ports:
      - "6379:6379"
```

```
docker compose up -d
docker ps            # check that redis is running
```

Default port is **6379**. Connection URL: `redis://localhost:6379`

6. Step 2: Try basic commands (redis-cli)

```
docker exec -it redis redis-cli
```

```
PING                      # → PONG (it works!)

SET name "Gourish"        # save a value
GET name                  # → "Gourish"
DEL name                  # delete it
EXISTS name               # → 0 (not found)

SET otp "4521" EX 60      # save it for 60 seconds only
TTL otp                   # seconds left, e.g. 57
                          # (-2 = gone, -1 = no expiry)

INCR visits               # counter → 1, 2, 3 ...

HSET user:1 name "Gourish" city "Goa"
HGETALL user:1            # → name Gourish city Goa

KEYS *                    # list all keys (dev only, slow in prod)
FLUSHALL                  # delete everything (careful!)
```

Key naming convention: use colons, e.g. `user:1`, `product:25`, `otp:9876543210`. It keeps keys organised.

7. Step 3: Use Redis in NestJS as a cache

Install

```
npm install @nestjs/cache-manager cache-manager @keyv/redis
```

app.module.ts

```
import { Module } from '@nestjs/common';
import { CacheModule } from '@nestjs/cache-manager';
import KeyvRedis from '@keyv/redis';

@Module({
  imports: [
    CacheModule.registerAsync({
      isGlobal: true,                // usable in every module
      useFactory: () => ({
        stores: [new KeyvRedis('redis://localhost:6379')],
        ttl: 60_000,                 // default expiry: 60 sec (in ms)
      }),
    }),
    // ...TypeOrmModule, StudentPgModule, etc.
  ],
})
export class AppModule {}
```

student-pg.service.ts (cache-aside pattern)

```ts
import { Inject, Injectable } from '@nestjs/common';
import { CACHE_MANAGER } from '@nestjs/cache-manager';
import { Cache } from 'cache-manager';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Student } from './student.entity';

@Injectable()
export class StudentPgService {
  constructor(
    @InjectRepository(Student) private repo: Repository<Student>,
    @Inject(CACHE_MANAGER) private cache: Cache,
  ) {}

  async findOne(id: number) {
    const key = `student:${id}`;

    // 1. Check Redis first
    const cached = await this.cache.get<Student>(key);
    if (cached) return cached;                // fast path

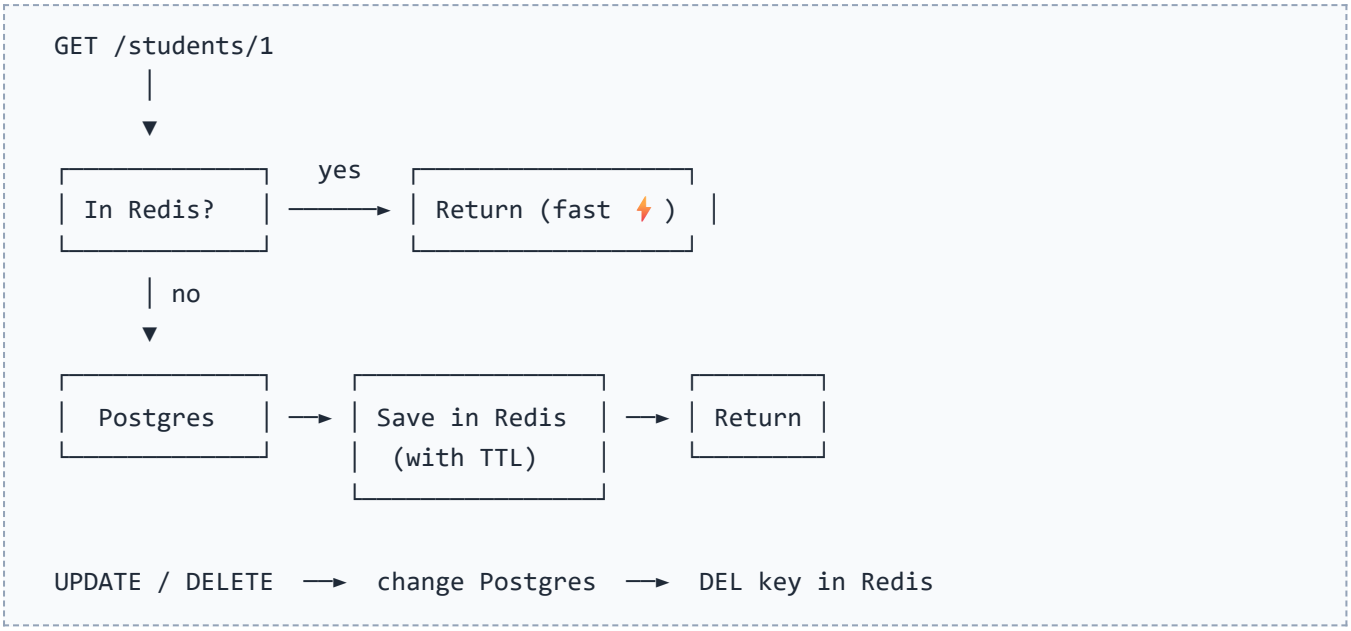
    // 2. Not in Redis, so ask Postgres
    const student = await this.repo.findOneBy({ id });

    // 3. Save it in Redis for next time (60 sec)
    if (student) await this.cache.set(key, student, 60_000);
    return student;
  }

  async update(id: number, dto: Partial<Student>) {
    await this.repo.update(id, dto);
    await this.cache.del(`student:${id}`);     // remove old copy!
    return this.findOne(id);
  }

  async remove(id: number) {
    await this.repo.delete(id);
    await this.cache.del(`student:${id}`);
  }
}
```

8. How cache-aside works



First request comes from Postgres (slower). **Next requests**, until the TTL runs out, come from Redis (very fast).

9. Shortcut: automatic caching with an interceptor

For simple GET routes you can let Nest cache the whole response:

```
import { CacheInterceptor, CacheTTL } from '@nestjs/cache-manager';
import { UseInterceptors } from '@nestjs/common';

@Controller('students')
@UseInterceptors(CacheInterceptor)       // caches GET responses
export class StudentPgController {

  @Get()
  @CacheTTL(30_000)                      // 30 seconds for this route
  findAll() {
    return this.service.findAll();
  }
}
```

> The interceptor only caches **GET** requests, and it does **not** clear the cache when data changes. For data that changes often, use the manual way (section 7).

10. Bonus: OTP example (expiring data)

```
// save OTP for 5 minutes
async sendOtp(phone: string) {
  const otp = Math.floor(1000 + Math.random() * 9000).toString();
  await this.cache.set(`otp:${phone}`, otp, 5 * 60_000);
  // ...send SMS
}

// verify
async verifyOtp(phone: string, otp: string) {
  const saved = await this.cache.get<string>(`otp:${phone}`);
  if (saved !== otp) throw new BadRequestException('Invalid or expired OTP');
  await this.cache.del(`otp:${phone}`);   // one-time use
  return { verified: true };
}
```

There's no table and no cleanup job. Redis deletes the OTP after 5 minutes on its own.

11. Cache methods cheat sheet

Method	What it does
`await cache.get(key)`	Read a value (`undefined` if missing or expired)
`await cache.set(key, value, ttlMs)`	Save a value with an expiry time
`await cache.del(key)`	Delete one key
`await cache.clear()`	Delete everything

> **TTL units:** In `redis-cli`, `EX` is in **seconds**. In NestJS cache-manager (v5+), TTL is in **milliseconds**. `60_000` = 60 seconds.

12. Golden rules

1. **Always set a TTL** so old data cleans itself up.
2. **Delete the cache key when data changes** (update or delete), or users will see old data.
3. **Don't keep important data only in Redis.** Postgres or MongoDB is the source of truth.
4. **Cache what is read often and changes rarely**, like product lists, user profiles, or settings.
5. **Don't cache everything.** Caching adds complexity, so only cache slow or frequent queries.
6. Keep the Redis URL in `.env` (`REDIS_URL=redis://localhost:6379`), not hard-coded.

13. Quick summary

Question	Answer
What is Redis?	An in-memory key-value store that is very fast
Why use it?	Speed: cache, OTP, sessions, rate limits, queues
Port	6379
Run locally	`docker run -d -p 6379:6379 redis:7`
NestJS packages	`@nestjs/cache-manager cache-manager @keyv/redis`
Main pattern	Cache-aside: check Redis → else DB → save in Redis
Biggest mistake	Forgetting to delete the cache after an update